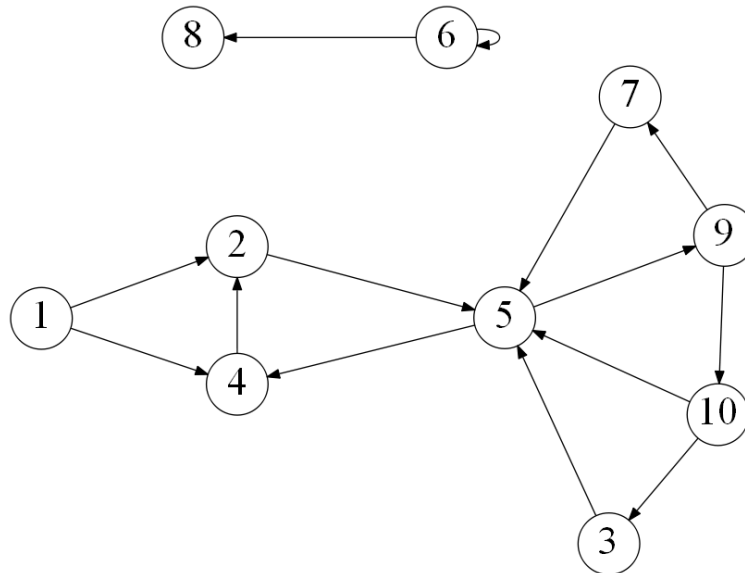


Name: Octavio MoralesDate: 10/19/2022

Point values are assigned for each question.

Points earned: ____ / 100

Consider the following graph:



1. Draw how the graph would look if represented by an adjacency matrix. You may assume the indexes are from 1 through 10. Indicate 1 if there is an edge from vertex A $\rightarrow$ vertex B, and 0 otherwise. (10 points)

	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>	<u>9</u>	<u>10</u>
<u>1</u>	0	1	0	1	0	0	0	0	0	0
<u>2</u>	0	0	0	0	1	0	0	0	0	0
<u>3</u>	0	0	0	0	1	0	0	0	0	0
<u>4</u>	0	1	0	0	0	0	0	0	0	0
<u>5</u>	0	0	0	1	0	0	0	0	1	0
<u>6</u>	0	0	0	0	0	1	0	1	0	0
<u>7</u>	0	0	0	0	1	0	0	0	0	0
<u>8</u>	0	0	0	0	0	0	0	0	0	0
<u>9</u>	0	0	0	0	0	0	1	0	0	1
<u>10</u>	0	0	1	0	1	0	0	0	0	0

2. Draw how the graph would look if represented by an adjacency list. You may assume the indexes are from 1 through 10. (10 points)

3. List the order in which the vertices are visited with a breadth-first search. If there are multiple vertices adjacent to a given vertex, visit the adjacent vertex with the lowest value first. (10 points)

Breadth-First Search- 1, 2, 4, 5, 9, 7, 10, 3, 6, 8

4. List the order in which the vertices are visited with a depth-first search. If there are multiple vertices adjacent to a given vertex, visit the adjacent vertex with the lowest value first. (10 points)

Depth-First Search- 1, 2, 5, 4, 9, 7, 10, 3, 6, 8

5. a) What is the running time of breadth-first search with an adjacency matrix? (5 points)

b) What is the running time of breadth-first search with an adjacency list? (5 points)

a. $O(V^2) = O(10^2) = O(100)$

b. $O(V + E) = O(10 + 14) = O(24)$

6. a) What is the running time of depth-first search with an adjacency matrix? (5 points)

b) What is the running time of depth-first search with an adjacency list? (5 points)

a. $O(V^2) = O(10^2) = O(100)$

b. $O(V + E) = O(10 + 14) = O(24)$

7. While an adjacency matrix is typically easier to code than an adjacency list, it is not always a better solution. Explain when an adjacency list is a clear winner in the efficiency of your algorithm? (5 points)

Adjacency lists are efficient when there is a large number of edges and fewer vertices, as shown by the run time formulas $O(V^2)$ and $O(V + E)$. This is because the number of edges won't affect the run time of the adjacency matrix.

8. Explain how one can use a breadth-first to determine if an undirected graph contains a cycle. (10 points)

Breadth-first search is able to determine if an undirected graph contains a cycle because it keeps track of the vertices it was gone through before to avoid repetition. If it does contain a cycle, the breadth-first search will have the option to loop back to itself.

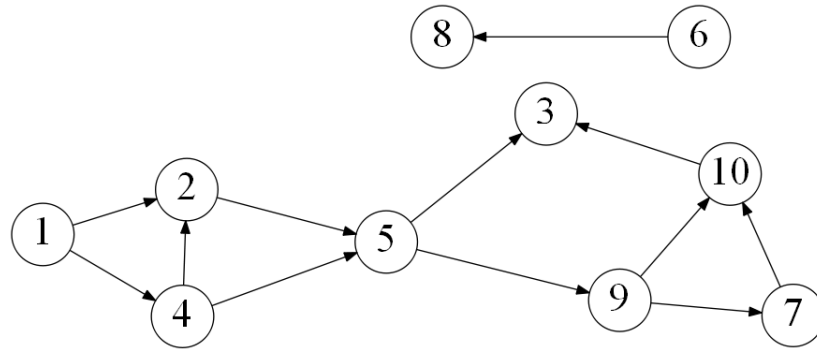
9. On undirected graphs, does either of the two traversals, DFS or BFS, always find a cycle faster than the other? If yes, indicate which of them is better and explain why it is the case; if not, draw two graphs supporting your answer and explain the graphs. (10 points)

DFS is typically faster on undirected graphs because it traverses through the stems of vertices rather than going through each stem possibility that breadth-first sort does.

10. Explain why a topological sort is not possible on the graph at the very top of this document. (5 points)

Topological sort is not possible on the graph because vertex 6 is pointing back to itself, so when breaking ties between itself 8 it, it would go back to 6.

Consider the following graph:



11. List the order in which the vertices are visited with a topological sort. Break ties by visiting the vertex with the lowest value first. (10 points)

6, 8, 1, 4, 2, 5, 9, 7, 10, 3